Відокремлений структурний підрозділ
«Економіко-технологічний фаховий коледж
Херсонського національного технічного університету»

ПРАКТИЧНА РОБОТА № 6

з дисципліни «Проєктний практикум»

Тема: «Тестування, виправлення дефектів і підготовка релізу»

Проєкт «EasyStock»

Виконав: Бондаренко Сергій
студент групи ЗІПЗс-24

Перевірив:
Дмитро Олегович Чорний
Ілля Володимирович Костевич

Хмельницький 2026

Мета та підсумок роботи

Я перевірив EasyStock за вимогами лабораторної №6, відтворив повторне проведення складської операції та усунув його для запитів із ключем повтору. Після виправлення пройдено 63 API-тести, 18 HTTP/React DOM і 8 релізних перевірок. Підготовлено локальний навчальний реліз v1.0.0 з інструкціями запуску, оновлення та rollback.

Виконав: Бондаренко Сергій. Робота індивідуальна: аналіз, розроблення, тестування, самоперевірку та документацію поєднує один автор. Адміністратор, комірник і менеджер є ролями застосунку. Незалежної перевірки іншою людиною не проводили.

Репозиторій EasyStock — github.com/chaek438/EasyStock

Параметр	Зафіксоване значення
Дата виконання	01.11.2026
Базова версія GitHub	0eabc81b33955948421838e0a98db920a799a640
Перевірений локальний коміт коду	6d7f8d951a23f205477960efdc3ec79648f6bb78
Платформа	Linux 6.18.57 x86_64; Python 3.12.14; SQLite 3.53.1
Backend	Flask 3.1.3; SQLAlchemy 2.0.48; Waitress 3.0.2; pytest 9.0.2
Frontend	Node.js 24.22.1; React 19.3.0; Vite 7.3.6; jsdom 30.1.2
Дані	Окремі тимчасові бази; деморолі admin, keeper, manager; без робочої БД користувача
Стан релізу	GitHub: PR №3, main і реліз v1.0.0; початкова локальна історія — у Git bundle

Обидва надані викладачем PDF описують одну лабораторну №6. У комплекті подання є звіт, тест план, повний каталог сценаріїв, журнал дефекту, результати перевірок, release notes, проєкт і відтворювана Git-історія.

Тест план і зв’язок із ризиками

У межах перевірки — авторизація, ролі, каталог, складські операції, пошук, аудит, збереження після перезапуску та додавання таблиці для повторів. Першими перевірено доступ, атомарність, повтор критичного запису і конкурентне списання.

Вимога	Ризик	Сценарії
US-AUTH і US-ROLES — вхід, вихід, серверні права	R3 доступ і конфігурація	01, 02, 09, 10, 17
US-PRODUCT — каталог, SKU, валідація	R1 цілісність даних	04–06, 12
US-STOCK — залишки, документи й аудит	R1 і R6 регресія	03, 11, 13, 15, 19
US-RETRY — один намір має один ефект	R1 повторна зміна	07, 08, 14, 16, 20
US-SEARCH — пошук та фільтри	R1 правильні дані	18
US-PERSIST — рестарт і оновлення схеми	R2 середовище; R7 тестові дані	21, 22
US-CONFIG — запуск без стандартних акаунтів	R3 конфігурація	25

Критерії початку та завершення

Вхідні критерії: зафіксована базова версія, встановлені залежності, готовий frontend, доступні тестові ролі та окрема база. Початкові 40 API-тестів пройшли. Вихідні критерії: критичні виконані сценарії Pass, DEF-01 пройшов retest, build і регресія успішні, відомі обмеження та rollback описані.

Межі автоматизації

Використано класи еквівалентності, граничні значення, таблицю дозволів і переходи станів операції та сеансу. Окремі тести не залежать від порядку виконання. Скрипти створюють власний Waitress на випадковому loopback-порту та видаляють тимчасові бази після завершення.

Поза підтвердженим обсягом — Chromium/Firefox, Windows нового коміту, незалежний людський review, навантаження багатьох користувачів, довільні міграції колонок та виробниче розгортання. Ці межі не прирівнюються до успішних перевірок.

Тестові сценарії від 1 до 8

Результати зафіксовано 01.11.2026. Повні передумови, дані, кроки, очікування, зв’язки з вимогами та назви тестів наведено у docs/Lab6/TestCases.md і TestCases.json. Позначення Pass означає відповідність конкретному очікуванню.

ID	Тип	Що перевірено	Фактичний результат	Статус
TC-01	Позитивний	Вхід і вихід	200; admin; після виходу 401.	Pass
TC-02	Негативний	Неправильний пароль	401; доступ не надано.	Pass
TC-03	Негативний	Нульова кількість	400; залишок 17, нових записів немає.	Pass
TC-04	Граничний	Мінімальна назва товару	201; один символ прийнято.	Pass
TC-05	Граничний	Межа довжини назви	200 прийнято; 201 відхилено.	Pass
TC-06	Позитивний	Створення і редагування товару	201; залишок 0; ціна 24.5.	Pass
TC-07	Бізнес правило	Повтор надходження	37; одна відповідь і один ефект.	Pass
TC-08	Бізнес правило	Повтор списання	14; один новий рух.	Pass

TC-07 і TC-08 мали Fail до виправлення: залишки 57 замість 37 і 11 замість 14. Обидва пов’язані з DEF-01. Після зміни вони Pass; початковий невдалий результат збережено в evidence/defect-before.txt.

Тестові сценарії від 9 до 16

Результати зафіксовано 01.11.2026. Повні передумови, дані, кроки, очікування, зв’язки з вимогами та назви тестів наведено у docs/Lab6/TestCases.md і TestCases.json. Позначення Pass означає відповідність конкретному очікуванню.

ID	Тип	Що перевірено	Фактичний результат	Статус
TC-09	Доступ	Запит без автентифікації	401; дані не видано.	Pass
TC-10	Доступ	Заборонена операція менеджера	200 для читання; 403 для запису.	Pass
TC-11	Негативний	Списання понад залишок	409; залишок 37; аудит незмінний.	Pass
TC-12	Негативний	Повторний SKU	201, потім 409.	Pass
TC-13	Відновлення	Недоступність БД під час запису	503 і повний rollback; повтор 201, залишок 19.	Pass
TC-14	Відновлення	Втрата відповіді після commit	Поле 17 збережене; ключ повторено; 17 + 20 = 37.	Pass
TC-15	Конфлікт	Конкурентні незалежні списання	201 і 409; залишок 2.	Pass
TC-16	Конфлікт	Одночасні дублікати одного запиту	Один ефект: 20 або 14; один ключ.	Pass

Для TC-14 транспорт тесту відкидає відповідь лише після реального HTTP commit. Це перевіряє повтор користувацької дії з тим самим ключем. Фізичне від’єднання мережі або повну браузерну сесію цей тест не моделює.

Тестові сценарії від 17 до 25

Результати зафіксовано 01.11.2026. Повні передумови, дані, кроки, очікування, зв’язки з вимогами та назви тестів наведено у docs/Lab6/TestCases.md і TestCases.json. Позначення Pass означає відповідність конкретному очікуванню.

ID	Тип	Що перевірено	Фактичний результат	Статус
TC-17	Доступ	CSRF перевіряється і для повтору	403; змін немає.	Pass
TC-18	Позитивний	Пошук і фільтри	Unicode та SKU знайдено; % без збігів; пагінація коректна.	Pass
TC-19	Граничний	Довжина коментаря	500 прийнято; 501 відхилено без зміни складу.	Pass
TC-20	Негативний	Один ключ для різних намірів	409; залишок 19.	Pass
TC-21	Дані	Повний перезапуск сервера	Після рестарту повтор не додає рух; сценарій 17 → 37 → 34 збережено.	Pass
TC-22	Сумісність даних	Оновлення попередньої схеми	Таблиця додана; старі дані та залишок 17 збережені.	Pass
TC-23	Сумісність	Chromium Firefox і Windows	Not run: перевірку цих браузерів і Windows для нової версії не виконано.	Not run
TC-24	Перехресна	Незалежна перевірка	Not run: робота індивідуальна; чужих висновків і схвалень немає.	Not run
TC-25	Безпека	Запуск без демоакаунтів	users=0; демоакаунти не створено.	Pass

Підсумок каталогу: 23 виконані сценарії Pass і 2 Not run. 63 API-приклади та 18 + 8 системних перевірок є деталізацією доказів, тому їх не додають до 23 як нові незалежні сценарії.

Журнал дефекту та відтворення

DEF 01 Повтор операції змінює залишок удруге

На базовій версії сервер ігнорував ідентифікатор наміру. Умова прояву — повтор складського POST після невідомого результату, наприклад втрати відповіді. Це одна проблема з двома відтвореними проявами: надходження і списання.

Поле	Зафіксований факт
Версія	Базова 0eabc81b3395; відтворення у локальному коміті ce14d0a
Передумови	Свіжа демобаза, SSD-480 має 17 одиниць, авторизований admin/keeper
Дані та дії	Двічі POST receipt: product_id=1, quantity=20, один ключ; окремо двічі writeoff: quantity=3
Очікування	Одна операція й один рух; надходження дає 37, списання дає 14
Факт до зміни	Два HTTP 201; надходження дає 57, списання дає 11; два тести Fail
Severity і priority	Major / High: неправильний складський облік після звичайного повтору; виправлення потрібне до релізу
Статус	Closed після власного retest; незалежного схвалення немає

Доказ до виправлення

```
receipt-20-37: assert 57 == 37  ⟶ FAIL
writeoff-3-14: assert 11 == 14  ⟶ FAIL
2 failed in 1.03s
```

Життєвий цикл: New → Triaged → In progress → Ready for retest → Verified → Closed. Усі ролі виконував один автор. Журнал і докази: docs/Lab6/Defects.md. Зміни оформлено в PR №3; окремий GitHub issue не створювався.

Виправлення та самоперевірка

Я додав збереження ключа запиту в ту саму SQLite транзакцію, що змінює залишок, документ та аудит. Унікальність пари user_id/request_key серіалізує одночасні дублікати. SHA-256 канонічного JSON і тип операції відрізняють повтор одного наміру від іншої дії.

Компонент	Зміна	Перевірка
models.py	Нова таблиця stock_requests: автор, ключ, тип, hash, відповідь і час	Один ключ для одного автора; нова таблиця додається без зміни старих даних
stock_service.py	Резервування ключа перед зміною складу; відповідь і склад commit разом	Повтор 201 без другого ефекту; інший намір 409; помилка не залишає ключ
routes.py	Передавання Idempotency-Key після серверної перевірки ролі й CSRF	Відомий ключ не обходить 401/403
React Operation	Ключ зберігається для повтору незміненого JSON після транспортної помилки	Після втрати відповіді поле збережене, ключ той самий, залишок не подвоєний

Правила протоколу

Новий намір отримує новий ключ з 1–128 дозволених ASCII-символів. Кеш прив’язаний до автора, а не сеансу, тому повтор працює після нового входу. Перевірка ролі та CSRF передує читанню кешу. Якщо транзакція відхилена, ключ не фіксується. Без заголовка зберігається попередній API-контракт, тому старий клієнт має додати ключ для безпечного автоматичного повтору.

Git і review

Гілка fix/lab6-stock-retry містить окремий commit із падаючими прикладами та commit виправлення 459101d8c30f. Локальна main об’єднує перевірені зміни й документацію. Самоперевірка записана у Review.md. Незалежного reviewer, другого автора чи фіктивних схвалень не додавали.

Повторне та регресійне тестування

Retest повторив умови DEF-01 і перевірив суміжні ситуації: одночасні дублікати, інший автор, новий сеанс, змінений JSON, неправильний ключ, rollback та межі полів. Регресія перевірила вже наявні API, потік складського обліку, ролі, збереження та React DOM.

Етап	Результат	Доказ у docs/Lab6/evidence
Базовий набір	40 Pass	baseline-api.txt та .xml
Відтворення DEF-01	2 Fail до зміни	defect-before.txt та .xml
Retest і нові випадки	23 Pass	retest.txt та .xml
Повна API-регресія	63 Pass	regression-api.txt та .xml
Збірка інтерфейсу	Vite build Pass	build.txt
HTTP і React DOM	18 перевірок Pass	smoke-ui.txt та .json
Релізні перевірки	8 перевірок Pass	release-smoke.txt та .json

Перевірка збереження й оновлення

Реальний процес Waitress завершено і запущено заново. Старий keyed запит після нового входу повернув початковий JSON без нового документа чи руху. Окремо виконано запуск бази попередньої структури без stock_requests: таблиця додалася, залишок 17 та старий аудит збереглися. Запуск без --demo створив таблиці, але кількість стандартних користувачів залишилася 0.

Відтворення

```
.venv/bin/python -m pytest -q
.venv/bin/python scripts/smoke_mvp.py --with-ui --json smoke.json
.venv/bin/python scripts/verify_release.py --json release.json
```

Для Windows використовують .venv\Scripts\python.exe. React DOM потребує npm-залежностей. Скрипт релізної перевірки також пройшов 8 перевірок без команди Git у PATH: zip-without-git.json. Тимчасові бази очищені; робочу базу користувача не відкривали.

Базова перевірка безпеки

Перевірки охопили серверні дозволи, CSRF, хешування паролів, закінчення і відкликання сеансів, відмови API без внутрішніх деталей, rollback, ігнорування приватних файлів та пошук характерних форматів секретів у текстових Git-об’єктах.

Роль	Каталог і історія	Складські операції	Користувачі
Без входу	401	401	401
Менеджер	200	403	403
Комірник	200	Дозволено із CSRF	403
Адміністратор	200	Дозволено із CSRF	Дозволено із CSRF

Запит із відомим Idempotency-Key без входу, з роллю менеджера або без CSRF також відхиляється. Інжекція OperationalError дає 503 зі зрозумілим повідомленням; внутрішня назва тестової помилки не потрапляє у відповідь. Дані та ключ повертаються до стану перед транзакцією.

Конфігурація і Git

Перевірено 57 commits і 185 унікальних Git blobs; 81 бінарних blobs виключено зі сканування вмісту. За застосованими шаблонами приватних ключів, GitHub-токенів, AWS access ID та ключів формату OpenAI знахідок немає. Поточний індекс не містить .env, робочої БД або Python-кешу; правила ігнорування перевірені. Це обмежений пошук, а не доказ відсутності будь-якого можливого секрету.

Демонстраційні облікові дані є відкритою частиною локального навчального режиму. Вони не застосовуються до реального складу. Перевірка запуску без --demo підтвердила users=0; для власного середовища створюють адміністратора з власним паролем. Перед зовнішнім доступом потрібні HTTPS, захищені cookie та окрема перевірка конфігурації.

До доказів не включено реальних персональних даних, робочих cookie чи токенів. Сеансові значення у тексті початкового невдалого тесту замасковані; числа та статуси результатів збережені.

Реліз запуск та відновлення

Навчальний реліз v1.0.0 опубліковано в GitHub через PR №3. Виконуваний код перевірено на локальному коміті 6d7f8d951a23; його відповідність вебкомітам підтверджує SHA-256 у Release_Notes.md. Git bundle зберігає обидві історії. PR: https://github.com/chaek438/EasyStock/pull/3

Запуск

Потрібен Python 3.11 або новіший. Розпакувати EasyStock і запустити run_demo.bat у Windows або bash run_demo.sh у Linux/macOS. Відкрити http://127.0.0.1:5000. Готовий frontend/dist входить до проєкту; Node.js для такого запуску не потрібен. Для першого встановлення залежностей потрібна мережа. Скрипти відновлюють відсутній pip через ensurepip.

Оновлення SQLite

Зупинити сервер і створити резервну копію БД. Нова версія додає лише stock_requests через db.create_all; існуючі таблиці і дані не переписуються. Цей шлях перевірений на тестовій старій схемі. Для довільних змін колонок автоматична міграція не заявляється.

Rollback

Базовий commit: 0eabc81b3395; локальний тег lab6-baseline є в Git bundle. Перед поверненням зупинити записи й зберегти базу окремо. Додаткова таблиця може залишатися. Відновлення БД із копії потрібне лише за необхідності й втрачає пізніші зміни. Після запуску перевірити health, вхід, залишки та історію на копії даних.

```
git switch --detach 0eabc81b3 395 594 842 1838e0a98db920a799a640
# Після перевірки повернутися до релізу
git switch --detach v1.0.0
```

Старий код має DEF-01, тому після rollback не можна повторювати невідомий POST без перевірки історії. Release_Notes.md містить можливості, виправлення, команди, межі й порядок відновлення; Publish.md описує опубліковану версію та Git bundle.

Відомі обмеження та рішення щодо релізу

Браузери та платформи

Chromium/Firefox і Windows нового коміту мають Not run. React DOM не перевіряє піксельну верстку. Надані автором знімки від 18.10.2026 ілюструють попередній локальний запуск.

Незалежна перевірка

Робота виконана одним автором. Немає людського review іншою людиною або перехресної сесії; є зафіксована самоперевірка.

Межі повтору

Захист застосовується до незміненого keyed запиту того самого автора і типу. Без заголовка POST є новою операцією. Після перезавантаження форми UI втрачає ключ у пам’яті; за невідомого результату спочатку перевіряють історію.

Зберігання ключів

Автоматичне очищення stock_requests не реалізовано. Потрібно контролювати розмір бази й резервних копій; видалення ключів змінює гарантію повтору для старих запитів.

Виробниче середовище

SQLite і limiter одного процесу не підтверджені для багатосерверної роботи. Зовнішні інтеграції, CSV-експорт і широке навантаження не входять до перевіреного обсягу.

Онлайн публікація

Код, звіт і докази додано до GitHub через PR №3. Реліз v1.0.0 містить готові файли. Початкові локальні commits тестів збережено окремо в Git bundle; вебкоміти мають інші SHA. Відповідність коду перевіряється за SHA-256.

Серед зареєстрованих результатів немає відкритих Blocker або Critical. DEF-01 закритий після retest. Версію підготовлено для локального навчального показу з наведеними межами; готовність усіх браузерів чи виробничого складу не стверджується.

Контрольні запитання та висновок

Які ризики покриває тест план

Повтор і конкурентна зміна залишків, розбіжність складу й аудиту, обхід ролі та CSRF, втрата даних після рестарту, помилки запуску й оновлення схеми. Пріоритет визначений впливом на критичний потік.

Чим відрізняються типи сценаріїв

Позитивний підтверджує дозволену дію з коректними даними; негативний очікує контрольованої відмови; граничний перевіряє значення біля межі, наприклад 200 і 201 символ назви.

Чому DEF 01 має Major та High

Він спотворює важливі складські дані після повтору, але не блокує весь застосунок і не доводить обхід доступу. Високий пріоритет зумовлений можливістю прояву під час звичайного повтору після втрати відповіді.

Що виконують після виправлення

Спочатку retest початкових кроків, потім перевірки суміжних правил і повну регресію: API, права, дані, збереження, запуск, HTTP та React DOM. Початковий Fail і новий Pass залишаються доступними.

Як перевіряють секрети і доступ

Виконують прямі API-запити без входу, з іншою роллю і без CSRF, аналізують безпечні помилки, поточний Git-індекс та історію за визначеними шаблонами. Такий набір знижує ризик, але не є повним доказом відсутності всіх секретів.

Які межі залишилися у релізі

Неперевірені браузери й Windows, відсутній незалежний review, ключ лише в пам’яті форми, старі API-клієнти без ключа та відсутність автоматичного очищення ключів. Їхній вплив і порядок дій описані у release notes.

Я пов’язав вимоги й ризики зі сценаріями та фактичними доказами, відтворив DEF-01 до зміни і підтвердив виправлення retest та регресією. Результат — локальний відтворюваний навчальний реліз із тестами, звітом, журналом дефекту, інструкціями й чесно зазначеними обмеженнями.

Скріншоти локального застосунку

Оригінальні знімки надав автор 18.10.2026. Вони ілюструють інтерфейс та демонстраційні дані попереднього запуску; результати нових перевірок підтверджуються журналами від 01.11.2026.

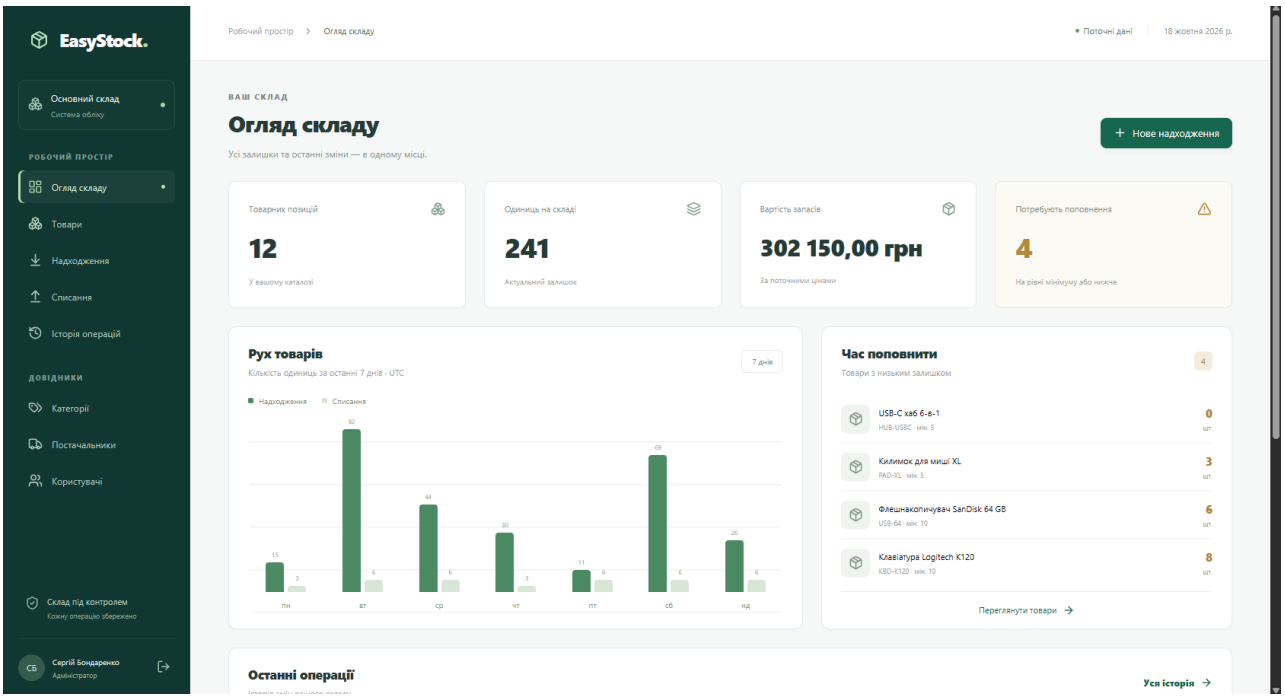


Рисунок 1 Огляд складу з демонстраційними показниками

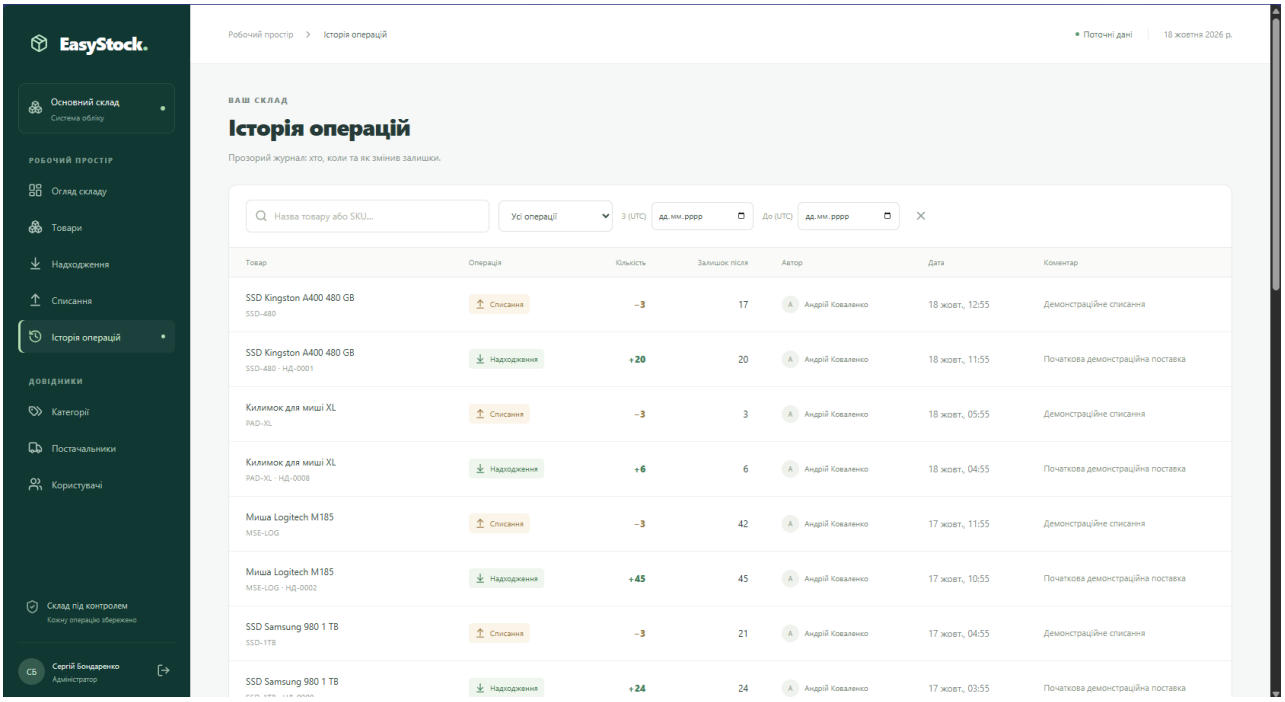


Рисунок 2 Історія складських операцій з автором і залишком

Повна галерея оригінальних скріншотів